CHATGPT 對話紀錄

GPT 輔助內容

HW2 Write Regression Feature Selection Evaluation

來源：https://chatgpt.com/share/6adc13c5-14c4-83ee-8148-be50f4d4fad7

整理日期：2026-10-24

內容：2 組使用者與 GPT 的可見對話訊息

本文件依 ChatGPT 分享連結整理，保留使用者與 GPT 的可見訊息；程式碼、表格與數學式依原始對話內容呈現。

使用者　2026-10-17 22:13

write sklearn to solve miltiple linear regression to solve Flood Prediction kaggle problem, following crisp-dm framework 必須執行 **特徵選擇 (Feature Selection)** 與 **模型評估 (Model Evaluation)**，結果部分需包含預測圖(加上信賴區間或預測區間) For Feature Selection, compare **SelectKBest, RFE, Sequential Feature Selection, Lasso and permutation importance**, selecting 14, 20, 25 or 30 features using training data only.

GPT　2026-10-17 22:13

Flood Prediction Kaggle：Multiple Linear Regression with Scikit-learn + CRISP-DM

這份實作會依照 **CRISP-DM 六階段流程**，建立一套可執行、可比較、可評估的多元線性迴歸（Multiple Linear Regression）實驗，並且完整執行你指定的五種 Feature Selection 方法。

設計重點：

- 使用 Scikit-learn 建立完整的 preprocessing 與 regression pipeline。
- 比較 `SelectKBest`、`RFE`、`Sequential Feature Selection`、`Lasso`、`Permutation Importance`。
- 從 **14、20、25、30 個特徵**中選擇，所有特徵選擇只使用 training data。
- 以交叉驗證比較模型，並使用獨立 test set 做最終評估。
- 產生實際值與預測值比較圖、殘差圖，以及具有 **95% 預測區間（Prediction Interval）**的預測圖。
- 輸出模型評估表、特徵排名、預測結果與圖表。

以下以 Kaggle 常見的 Flood Prediction / Flood Prediction Dataset 為目標資料集。這個版本的競賽使用 `FloodProbability` 作為預測目標，屬於監督式迴歸問題。若你使用的是其他 Flood Prediction Kaggle dataset，只需要調整資料路徑與 target column 設定。

1. CRISP-DM 實驗架構

1. Business Understanding

預測洪水機率，將迴歸誤差作為主要評估依據。

2. Data Understanding

載入 CSV、確認欄位、缺失值、重複資料、target 分布及資料洩漏風險。

3. Data Preparation

切分 train/test、建立 imputation 與 scaling pipeline；測試集不參與特徵選擇。

4. Modeling

五種特徵選擇方法 × 四種特徵數量 × Multiple Linear Regression。

5. Evaluation

交叉驗證 RMSE、測試集 RMSE、MAE、R^2、預測區間覆蓋率。

6. Deployment

保存最佳模型、特徵清單、比較結果與預測圖表。

2. Dataset 與 Feature Selection 設計

Kaggle 的 S4E5 資料集包含 20 個主要預測特徵，目標欄位為 `FloodProbability`，官方競賽評估指標是 `R^2`。

因此，若要求比較 14、20、25、30 個特徵，不能直接從原始的 20 個欄位選出 25 或 30 個。以下程式會建立 **30 個候選特徵**：

- 20 個原始數值特徵。
- 10 個事先定義的交互作用特徵，例如 `MonsoonIntensity × ClimateChange`。

這讓四種特徵數量都能公平比較，同時保持線性迴歸模型的形式：對新增交互作用特徵而言，模型仍是線性參數模型。

Feature Selection 方法	選擇原理
SelectKBest	依據單變量 F-test 選出前 `k` 個特徵
RFE	反覆移除不重要的特徵
Sequential Feature Selection	以逐步加入特徵的方式尋找子集合
Lasso	利用 L1 正則化係數，再依係數絕對值選取前 `k` 個
Permutation Importance	衡量打亂單一特徵後，模型表現下降多少

實驗的四種特徵數量為 `k \in \{14,20,25,30\}`，合計最多 20 組方法與特徵數量的組合。

3. 安裝環境

在 Jupyter Notebook、VS Code 或 Kaggle Notebook 執行：

```
pip install numpy pandas matplotlib seaborn scikit-learn joblib
```

將 Kaggle 的 `train.csv`、`test.csv` 與 `sample_submission.csv` 放在 `data/` 目錄下。官方資料與競賽說明可由 Kaggle Dataset 查看。

4. 完整 Scikit-learn 程式碼

建議將以下程式放在一個 Notebook 中依序執行。程式會保留獨立測試集，另外切分驗證集與校準集；所有特徵選擇只會讀取模型訓練集的資料與標籤。

Step 1 — Imports and configuration

```
from pathlib import Path
import json
import time
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.base import clone
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import (
    SelectKBest,
    f_regression,
    RFE,
    SequentialFeatureSelector,
)
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.inspection import permutation_importance
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)

warnings.filterwarnings("ignore", category=FutureWarning)

RANDOM_STATE = 42
TARGET = "FloodProbability"
FEATURE_COUNTS = [14, 20, 25, 30]

DATA_DIR = Path("data")
OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

TRAIN_PATH = DATA_DIR / "train.csv"
TEST_PATH = DATA_DIR / "test.csv"
SUBMISSION_PATH = DATA_DIR / "sample_submission.csv"
```

Step 2 — Data understanding and preparation

```
train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)

assert TARGET in train_df.columns, (
```

```
    f"Target '{TARGET}' not found. "
    f"Available columns: {train_df.columns.tolist()}"
)

print("Training shape:", train_df.shape)
print("Test shape:", test_df.shape)
print("\nMissing values:")
print(train_df.isna().sum().sort_values(ascending=False).head(10))

print("\nTarget summary:")
print(train_df[TARGET].describe())

# The ID is an identifier, not a predictive feature.
ID_COL = "id" if "id" in train_df.columns else None

base_features = [
    c for c in train_df.columns
    if c != TARGET and c != ID_COL
]

# This implementation expects the numerical Kaggle S4E5 dataset.
non_numeric = train_df[base_features].select_dtypes(
    exclude=np.number
).columns.tolist()

if non_numeric:
    raise ValueError(
        f"Categorical columns need encoding first: {non_numeric}"
    )

X_raw = train_df[base_features].copy()
y = train_df[TARGET].copy()

print("Original feature count:", len(base_features))
print("Duplicate training rows:", train_df.duplicated().sum())

fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(y, kde=True, ax=ax)
ax.set_title("Distribution of FloodProbability")
ax.set_xlabel(TARGET)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "target_distribution.png", dpi=150)
plt.show()
```

Step 3 — Feature engineering: create 30 candidate features

交互作用的定義在看到測試集之前就固定下來，不依據測試集的標籤進行選擇。

```
# Ten predefined interactions. The code checks that all source
# columns exist, so it will fail clearly for a different dataset.

INTERACTIONS = [
    ("MonsoonIntensity", "ClimateChange"),
    ("MonsoonIntensity", "DrainageSystems"),
    ("MonsoonIntensity", "RiverManagement"),
    ("ClimateChange", "Deforestation"),
    ("ClimateChange", "Urbanization"),
    ("Deforestation", "Urbanization"),
    ("DamsQuality", "DrainageSystems"),
    ("WetlandLoss", "CoastalVulnerability"),
```

```
    ("Watersheds", "Landslides"),
    ("PopulationScore", "InadequatePlanning"),
]

# Some dataset descriptions use different spellings for certain
# features. Map known aliases explicitly; do not silently guess.
ALIASES = {
    "WetlandLoss": ["WetlandLoss", "WetlandLosses"],
    "CoastalVulnerability": [
        "CoastalVulnerability", "CoastalVulnerabilities"
    ],
    "InadequatePlanning": [
        "InadequatePlanning", "InadequatePlanningScore"
    ],
}

def resolve_column(name, columns):
    candidates = ALIASES.get(name, [name])
    for candidate in candidates:
        if candidate in columns:
            return candidate
    raise ValueError(
        f"Cannot find '{name}'. Check the actual dataset columns: "
        f"{list(columns)}"
    )

def add_interactions(X):
    X = X.copy()

    for left, right in INTERACTIONS:
        left_col = resolve_column(left, X.columns)
        right_col = resolve_column(right, X.columns)

        name = f"{left_col}_x_{right_col}"
        X[name] = X[left_col] * X[right_col]

    return X

X_all = add_interactions(X_raw)

# Enforce the expected design: 20 original + 10 interactions.
assert X_all.shape[1] == 30, (
    f"Expected 30 candidates, got {X_all.shape[1]}. "
    "Check the source dataset and interaction definitions."
)

print("Candidate features:", X_all.shape[1])
print(X_all.columns.tolist())
```

Step 4 — Split data without leakage

這裡將資料分成：

- 60% fit/training set：訓練模型及執行特徵選擇。
- 15% validation set：比較不同方法及特徵數量。
- 10% calibration set：建立 95% 預測區間。
- 15% test set：最後一次的獨立模型評估。

```
# First isolate the final test set.
X_dev, X_test, y_dev, y_test = train_test_split(
    X_all, y,
    test_size=0.15,
    random_state=RANDOM_STATE,
)

# Split the remaining 85% into fit, validation, and calibration.
# 15% / 85% = 17.647% of the development data for validation.
X_fit, X_temp, y_fit, y_temp = train_test_split(
    X_dev, y_dev,
    test_size=0.3529411765,
    random_state=RANDOM_STATE,
)

# 10% of the original data for calibration:
# 10 / 25 = 40% of the remaining temporary data.
X_val, X_cal, y_val, y_cal = train_test_split(
    X_temp, y_temp,
    test_size=0.40,
    random_state=RANDOM_STATE,
)

print("Fit:", X_fit.shape)
print("Validation:", X_val.shape)
print("Calibration:", X_cal.shape)
print("Test:", X_test.shape)
```

資料洩漏原則： 特徵選擇器、Lasso、Permutation Importance 都只在 `X_fit, y_fit` 上擬合；`X_val` 用於模型選擇，`X_cal` 用於建立區間，`X_test` 不參與上述任何選擇。

Step 5 — Implement five feature selection methods

以下程式讓每個方法都輸出固定數量的特徵。對 Lasso 而言，先用 `LassoCV` 在訓練資料內選擇正則化強度，再依照係數絕對值排名，取前 `k` 個。這能滿足固定特徵數量的比較要求，即使 Lasso 本身產生的非零係數數量不是指定的 `k`。

```
def make_preprocessor():
    return Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

def make_selection_pipeline(selector):
    return Pipeline([
        ("preprocess", make_preprocessor()),
        ("selector", selector),
        ("regressor", LinearRegression()),
    ])

def fit_feature_selector(method, k, X_train, y_train):
    """
    Fit feature selection using training data only.
    Returns a list of selected feature names.
    """
    n_features = X_train.shape[1]
    k = min(k, n_features)
```

```
    preprocessor = make_preprocessor()
    X_scaled = preprocessor.fit_transform(X_train)

    if method == "SelectKBest":
        selector = SelectKBest(
            score_func=f_regression,
            k=k,
        )
        selector.fit(X_scaled, y_train)
        selected = selector.get_support()

    elif method == "RFE":
        selector = RFE(
            estimator=LinearRegression(),
            n_features_to_select=k,
            step=1,
        )
        selector.fit(X_scaled, y_train)
        selected = selector.get_support()

    elif method == "Sequential":
        # Forward SFS is computationally expensive. Use a fixed
        # training-only subset to keep runtime manageable.
        rng = np.random.RandomState(RANDOM_STATE)
        n_select = min(len(X_scaled), 5000)
        idx = rng.choice(len(X_scaled), n_select, replace=False)

        selector = SequentialFeatureSelector(
            estimator=LinearRegression(),
            n_features_to_select=k,
            direction="forward",
            scoring="neg_root_mean_squared_error",
            cv=2,
            n_jobs=-1,
        )
        selector.fit(X_scaled[idx], np.asarray(y_train)[idx])
        selected = selector.get_support()

    elif method == "Lasso":
        lasso = LassoCV(
            cv=3,
            random_state=RANDOM_STATE,
            max_iter=5000,
            n_jobs=-1,
        )
        lasso.fit(X_scaled, y_train)

        # Rank features by absolute coefficient magnitude.
        importance = np.abs(lasso.coef_)
        indices = np.argsort(importance)[::-1][:k]

        selected = np.zeros(n_features, dtype=bool)
        selected[indices] = True

    elif method == "Permutation":
        # Fit a baseline linear model using training data only.
        baseline = LinearRegression()
        baseline.fit(X_scaled, y_train)

        # Limit permutation evaluation to a fixed training subset.
        rng = np.random.RandomState(RANDOM_STATE)
```

```
        n_select = min(len(X_scaled), 10000)
        idx = rng.choice(len(X_scaled), n_select, replace=False)

        perm = permutation_importance(
            baseline,
            X_scaled[idx],
            np.asarray(y_train)[idx],
            scoring="neg_root_mean_squared_error",
            n_repeats=3,
            random_state=RANDOM_STATE,
            n_jobs=-1,
        )

        indices = np.argsort(perm.importances_mean)[::-1][:k]
        selected = np.zeros(n_features, dtype=bool)
        selected[indices] = True

    else:
        raise ValueError(f"Unknown method: {method}")

    return X_train.columns[selected].tolist()
```

Step 6 — Compare all methods and feature counts

為了讓比較更容易解讀，統一使用相同的 `LinearRegression` 作為最終預測模型。各方法的差異只在選出的特徵集合。

```
METHODS = [
    "SelectKBest",
    "RFE",
    "Sequential",
    "Lasso",
    "Permutation",
]

results = []
selected_features = {}
fitted_models = {}

for method in METHODS:
    for k in FEATURE_COUNTS:
        print(f"Running {method}, k={k} ...")
        start = time.time()

        features = fit_feature_selector(
            method, k, X_fit, y_fit
        )

        model = Pipeline([
            ("preprocess", make_preprocessor()),
            ("regressor", LinearRegression()),
        ])

        model.fit(X_fit[features], y_fit)
        val_pred = model.predict(X_val[features])

        rmse = np.sqrt(mean_squared_error(y_val, val_pred))
        mae = mean_absolute_error(y_val, val_pred)
        r2 = r2_score(y_val, val_pred)
```

```
        key = (method, k)
        selected_features[key] = features
        fitted_models[key] = model

        results.append({
            "method": method,
            "n_features": len(features),
            "requested_k": k,
            "validation_RMSE": rmse,
            "validation_MAE": mae,
            "validation_R2": r2,
            "runtime_seconds": time.time() - start,
        })

results_df = pd.DataFrame(results).sort_values(
    "validation_RMSE", ascending=True
)

results_df.to_csv(
    OUTPUT_DIR / "feature_selection_comparison.csv",
    index=False,
)

print(results_df.to_string(index=False))
```

Step 7 — Model evaluation and best model selection

Kaggle 官方使用 `R^2` 評估，但本實驗也比較 RMSE 與 MAE。以下用驗證集 RMSE 最低的組合作為最佳模型；測試集只在選擇完成後評估一次。

```
best_row = results_df.iloc[0]
best_key = (best_row["method"], int(best_row["requested_k"]))

best_method, best_k = best_key
best_features = selected_features[best_key]
best_model = fitted_models[best_key]

print("Best method:", best_method)
print("Number of features:", len(best_features))
print("Selected features:", best_features)

test_pred = best_model.predict(X_test[best_features])

test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))
test_mae = mean_absolute_error(y_test, test_pred)
test_r2 = r2_score(y_test, test_pred)

print(f"Test RMSE: {test_rmse:.6f}")
print(f"Test MAE : {test_mae:.6f}")
print(f"Test R²  : {test_r2:.6f}")

test_metrics = pd.DataFrame([{
    "best_method": best_method,
    "n_features": len(best_features),
    "RMSE": test_rmse,
    "MAE": test_mae,
    "R2": test_r2,
}])
```

```
test_metrics.to_csv(
    OUTPUT_DIR / "test_metrics.csv",
    index=False,
)
```

Step 8 — Generate 95% prediction intervals

使用 **Split Conformal Prediction** 建立預測區間。這個方法不要求殘差服從常態分布，而且能在校準資料與測試資料可交換等條件下，提供有限樣本的邊際覆蓋率保證。

對校準集計算絕對殘差：

```
r_i=|y_i-\hat y_i|
```

令 `\alpha=0.05`，使用有限樣本修正後的分位數：

```
q=r_{(\lceil(n+1)(1-\alpha)\rceil)}
```

則每筆預測的區間為：

```
[\hat y-q,\ \hat y+q]
```

```
ALPHA = 0.05

# The selected model was trained on X_fit.
# Calibration data was not used for model fitting or selection.
cal_pred = best_model.predict(X_cal[best_features])
cal_residuals = np.abs(np.asarray(y_cal) - cal_pred)

n_cal = len(cal_residuals)
rank = min(
    n_cal,
    int(np.ceil((n_cal + 1) * (1 - ALPHA))),
)

q_hat = np.sort(cal_residuals)[rank - 1]

lower = test_pred - q_hat
upper = test_pred + q_hat

coverage = np.mean(
    (np.asarray(y_test) >= lower) &
    (np.asarray(y_test) <= upper)
)

mean_width = np.mean(upper - lower)

print(f"Conformal quantile: {q_hat:.6f}")
print(f"Nominal coverage: {1 - ALPHA:.0%}")
print(f"Observed test coverage: {coverage:.4%}")
print(f"Mean interval width: {mean_width:.6f}")

interval_metrics = pd.DataFrame([{
```

```
    "nominal_coverage": 1 - ALPHA,
    "observed_test_coverage": coverage,
    "mean_interval_width": mean_width,
    "conformal_quantile": q_hat,
}])

interval_metrics.to_csv(
    OUTPUT_DIR / "prediction_interval_metrics.csv",
    index=False,
)
```

注意：這是**預測區間**，不是迴歸係數或平均預測值的信賴區間。上述區間在理想的可交換性條件下具有邊際覆蓋率保證，但不保證每一種洪水風險族群都具有相同覆蓋率。

Step 9 — Visualization

這個步驟產生三類結果圖：特徵選擇比較、預測區間圖，以及殘差分析圖。

9.1 Feature selection comparison

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.lineplot(
    data=results_df,
    x="n_features",
    y="validation_RMSE",
    hue="method",
    marker="o",
    ax=axes[0],
)
axes[0].set_title("Feature Selection vs Validation RMSE")
axes[0].set_xlabel("Number of Selected Features")
axes[0].set_ylabel("Validation RMSE")

sns.lineplot(
    data=results_df,
    x="n_features",
    y="validation_R2",
    hue="method",
    marker="o",
    ax=axes[1],
)
axes[1].set_title("Feature Selection vs Validation R²")
axes[1].set_xlabel("Number of Selected Features")
axes[1].set_ylabel("Validation R²")

plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "feature_selection_comparison.png",
    dpi=160,
    bbox_inches="tight",
)
plt.show()
```

9.2 Actual vs predicted with 95% prediction interval

測試資料可能有數十萬筆，直接繪製全部樣本會非常擁擠。因此，圖表隨機抽取最多 300 筆作為展示，但評估指標仍使用完整測試集。

```
plot_df = pd.DataFrame({
    "actual": np.asarray(y_test),
    "predicted": test_pred,
    "lower": lower,
    "upper": upper,
}).reset_index(drop=True)

plot_df["abs_error"] = np.abs(
    plot_df["actual"] - plot_df["predicted"]
)

plot_df = plot_df.sample(
    n=min(300, len(plot_df)),
    random_state=RANDOM_STATE,
).sort_values("predicted").reset_index(drop=True)

fig, ax = plt.subplots(figsize=(14, 6))

x = np.arange(len(plot_df))

ax.fill_between(
    x,
    plot_df["lower"],
    plot_df["upper"],
    alpha=0.25,
    label="95% Prediction Interval",
)

ax.plot(
    x,
    plot_df["actual"],
    ".",
    markersize=4,
    label="Actual",
)

ax.plot(
    x,
    plot_df["predicted"],
    "-",
    linewidth=1.5,
    label="Predicted",
)

ax.set_title(
    f"FloodProbability: Actual vs Predicted "
    f"with 95% Prediction Interval\n"
    f"{best_method}, k={best_k}"
)
ax.set_xlabel("Test samples sorted by predicted value")
ax.set_ylabel("FloodProbability")
ax.legend()

plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "prediction_interval.png",
```

```
    dpi=160,
    bbox_inches="tight",
)
plt.show()
```

9.3 Actual vs predicted parity plot and residual plot

```
residuals = np.asarray(y_test) - test_pred

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].scatter(
    y_test, test_pred,
    alpha=0.2, s=8,
)
vmin = min(float(np.min(y_test)), float(np.min(test_pred)))
vmax = max(float(np.max(y_test)), float(np.max(test_pred)))
axes[0].plot(
    [vmin, vmax], [vmin, vmax],
    "k--",
    label="Perfect prediction",
)
axes[0].set_xlabel("Actual FloodProbability")
axes[0].set_ylabel("Predicted FloodProbability")
axes[0].set_title(f"Actual vs Predicted (R²={test_r2:.4f})")
axes[0].legend()

axes[1].scatter(
    test_pred, residuals,
    alpha=0.2, s=8,
)
axes[1].axhline(0, linestyle="--", color="black")
axes[1].set_xlabel("Predicted FloodProbability")
axes[1].set_ylabel("Residual (Actual - Predicted)")
axes[1].set_title("Residual Analysis")

plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "regression_diagnostics.png",
    dpi=160,
    bbox_inches="tight",
)
plt.show()
```

解讀圖表時，請注意：

- **Parity plot：** 點越接近對角線，預測越準確。
- **Residual plot：** 若殘差出現明顯曲線、漏斗形或系統性偏差，可能代表線性模型的假設不足。
- **Prediction interval：** 真實值應有接近 95% 的比例落在區間內；若偏低，需檢查資料分布、校準樣本數及資料可交換性。

Step 10 — Save results and generate Kaggle submission

最後輸出比較結果、最佳特徵清單、模型與 Kaggle 提交檔案。

```
# Save the selected feature list and evaluation summary.
with open(
    OUTPUT_DIR / "best_features.json",
    "w",
    encoding="utf-8",
) as f:
    json.dump(
        {
            "method": best_method,
            "n_features": len(best_features),
            "features": best_features,
        },
        f,
        ensure_ascii=False,
        indent=2,
    )

joblib.dump(
    best_model,
    OUTPUT_DIR / "best_model.joblib",
)

# For Kaggle submission, refit the selected-feature model
# on all labeled training data. The feature list was chosen
# without using the held-out test labels.
X_submission = add_interactions(
    train_df[base_features].copy()
)
X_kaggle_test = add_interactions(
    test_df[base_features].copy()
)

submission_model = Pipeline([
    ("preprocess", make_preprocessor()),
    ("regressor", LinearRegression()),
])

submission_model.fit(
    X_submission[best_features],
    train_df[TARGET],
)

kaggle_pred = submission_model.predict(
    X_kaggle_test[best_features]
)

# Match the official sample submission structure.
if SUBMISSION_PATH.exists():
    submission = pd.read_csv(SUBMISSION_PATH)
    submission[TARGET] = kaggle_pred
else:
    submission = pd.DataFrame({
        ID_COL if ID_COL else "id": test_df[ID_COL]
        if ID_COL else np.arange(len(test_df)),
        TARGET: kaggle_pred,
    })

submission.to_csv(
    OUTPUT_DIR / "submission.csv",
    index=False,
)
```

```
joblib.dump(
    submission_model,
    OUTPUT_DIR / "submission_model.joblib",
)

print("Saved results to:", OUTPUT_DIR.resolve())
print("Submission shape:", submission.shape)
print(submission.head())
```

5. CRISP-DM 最終報告

程式執行完成後，可以使用下列表格整理你的實驗報告。

CRISP-DM 階段	應呈現的結果
Business Understanding	預測 `FloodProbability`，以 `R^2` 為官方主要評估指標
Data Understanding	資料筆數、特徵數量、缺失值與 target 分布
Data Preparation	20 個原始特徵 + 10 個交互作用特徵，明確隔離測試集
Modeling	五種 Feature Selection × 四種特徵數量
Evaluation	Validation RMSE、MAE、R^2；獨立 Test RMSE、MAE、R^2；95% 區間覆蓋率與平均寬度
Deployment	`submission.csv`、模型檔、特徵清單與所有圖表

建議在報告中放入以下四項主要成果：

1. `feature_selection_comparison.png`：比較五種特徵選擇方法與不同 `k` 值。
2. `prediction_interval.png`：實際值、預測值及 95% 預測區間。
3. `regression_diagnostics.png`：觀察模型是否存在系統性殘差。
4. `test_metrics.csv`：報告獨立測試集的最終模型表現。

重要限制： 這是一套用於課程實驗的多元線性迴歸流程，不保證是 Kaggle 競賽的最高分方法。這個資料集的合成特性可能使不同特徵之間具有高度相關性；因此，即使 `R^2` 很高，也不能直接推論這些特徵在真實洪水事件中具有因果關係。另請注意，Sequential Feature Selection 的運算成本較高，程式以訓練資料子樣本降低運算時間；若使用不同版本的資料集，請先確認 20 個原始欄位名稱與交互作用欄位是否一致。

使用者 2026-10-18 08:13

請把CRISP-DM 實驗架構的所有程式碼整合在一起

GPT 2026-10-18 08:13

以下將前面 CRISP-DM 六個階段的程式碼整合成一份完整的 Python 程式，方便直接存成 `flood_prediction.py` 或放進 Jupyter Notebook 執行。

整合版包含：

- **Business Understanding**：定義預測目標與評估指標。
- **Data Understanding**：資料檢查、缺失值、目標分布。
- **Data Preparation**：資料切分、前處理、30 個候選特徵。
- **Modeling**：比較 `SelectKBest`、`RFE`、`Sequential Feature Selection`、`Lasso`、`Permutation Importance`，每種方法測試 14、20、25、30 個特徵。
- **Evaluation**：交叉驗證式特徵選擇流程中的驗證集比較、獨立測試集評估、預測區間與三種圖表。
- **Deployment**：儲存最佳模型、特徵清單及 Kaggle submission。

我也調整了資料切分流程，避免測試集參與特徵選擇或模型挑選。以下程式採用單一驗證集比較 20 組候選組合，最後只評估一次獨立測試集。

完整整合程式碼：`flood_prediction.py`

先安裝必要套件：

```
pip install numpy pandas matplotlib seaborn scikit-learn joblib
```

資料夾建議如下：

```
flood_prediction/
├── data/
│   ├── train.csv
│   ├── test.csv
│   └── sample_submission.csv
├── outputs/
└── flood_prediction.py
```

將以下程式碼完整複製到 `flood_prediction.py`，確認資料路徑後即可執行。

```
# ============================================================
# Flood Prediction with Multiple Linear Regression
# CRISP-DM: Complete End-to-End Implementation
#
# Dataset: Kaggle Playground Series S4E5
# Target: FloodProbability
#
# Feature Selection:
#   1. SelectKBest
#   2. RFE
#   3. Sequential Feature Selection
#   4. Lasso
```

```
#   5. Permutation Importance
#
# Feature counts: 14, 20, 25, 30
# Model: Multiple Linear Regression
# Evaluation: RMSE, MAE, R2, Prediction Interval Coverage
# ============================================================

from pathlib import Path
import json
import time
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import (
    SelectKBest,
    f_regression,
    RFE,
    SequentialFeatureSelector,
)
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)

warnings.filterwarnings("ignore", category=FutureWarning)

# ============================================================
# Configuration
# ============================================================

RANDOM_STATE = 42
TARGET = "FloodProbability"
FEATURE_COUNTS = [14, 20, 25, 30]
ALPHA = 0.05

DATA_DIR = Path("data")
OUTPUT_DIR = Path("outputs")

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

TRAIN_PATH = DATA_DIR / "train.csv"
TEST_PATH = DATA_DIR / "test.csv"
SUBMISSION_PATH = DATA_DIR / "sample_submission.csv"

# ============================================================
# CRISP-DM 1: BUSINESS UNDERSTANDING
# ============================================================

print("\n" + "=" * 70)
print("CRISP-DM 1: BUSINESS UNDERSTANDING")
```

```
print("=" * 70)

print("Objective: Predict FloodProbability.")
print("Model: Multiple Linear Regression.")
print("Official competition metric: R2.")
print("Additional metrics: RMSE and MAE.")
print("Feature selection: 5 methods x 4 feature counts.")
print("Prediction intervals: 95% split conformal intervals.")

# ============================================================
# CRISP-DM 2: DATA UNDERSTANDING
# ============================================================

print("\n" + "=" * 70)
print("CRISP-DM 2: DATA UNDERSTANDING")
print("=" * 70)

train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)

if TARGET not in train_df.columns:
    raise ValueError(
        f"Target '{TARGET}' not found in train.csv."
    )

ID_COL = "id" if "id" in train_df.columns else None

base_features = [
    col for col in train_df.columns
    if col != TARGET and col != ID_COL
]

missing_test_features = [
    col for col in base_features
    if col not in test_df.columns
]

if missing_test_features:
    raise ValueError(
        f"Missing test features: {missing_test_features}"
    )

non_numeric = train_df[base_features].select_dtypes(
    exclude=np.number
).columns.tolist()

if non_numeric:
    raise ValueError(
        "This implementation expects numerical features. "
        f"Encode categorical columns first: {non_numeric}"
    )

X_raw = train_df[base_features].copy()
y = train_df[TARGET].copy()

print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
print("Original feature count:", len(base_features))
print("Duplicate training rows:", train_df.duplicated().sum())

print("\nMissing values:")
print(
```

```
    train_df[base_features + [TARGET]]
    .isna()
    .sum()
    .sort_values(ascending=False)
    .head(15)
)

print("\nTarget summary:")
print(y.describe())

# Target distribution plot
plt.figure(figsize=(8, 5))
sns.histplot(y, kde=True)
plt.title("Distribution of FloodProbability")
plt.xlabel(TARGET)
plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "target_distribution.png",
    dpi=160,
)
plt.close()

# ============================================================
# CRISP-DM 3: DATA PREPARATION
# ============================================================

print("\n" + "=" * 70)
print("CRISP-DM 3: DATA PREPARATION")
print("=" * 70)

# Ten predefined interaction features.
# Together with the 20 original predictors, these produce
# 30 candidate features for the requested comparison.

INTERACTIONS = [
    ("MonsoonIntensity", "ClimateChange"),
    ("MonsoonIntensity", "DrainageSystems"),
    ("MonsoonIntensity", "RiverManagement"),
    ("ClimateChange", "Deforestation"),
    ("ClimateChange", "Urbanization"),
    ("Deforestation", "Urbanization"),
    ("DamsQuality", "DrainageSystems"),
    ("WetlandLoss", "CoastalVulnerability"),
    ("Watersheds", "Landslides"),
    ("PopulationScore", "InadequatePlanning"),
]

def add_interactions(X):
    """Create predefined pairwise interaction features."""

    X = X.copy()

    for left, right in INTERACTIONS:
        if left not in X.columns:
            raise ValueError(
                f"Required interaction column missing: {left}"
            )

        if right not in X.columns:
            raise ValueError(
                f"Required interaction column missing: {right}"
            )
```

```
        X[f"{left}_x_{right}"] = X[left] * X[right]

    return X

X_all = add_interactions(X_raw)

if X_all.shape[1] < max(FEATURE_COUNTS):
    raise ValueError(
        f"Need at least {max(FEATURE_COUNTS)} candidate features, "
        f"but only {X_all.shape[1]} were created."
    )

print("Original predictors:", X_raw.shape[1])
print("All candidate predictors:", X_all.shape[1])

# Split 15% for final testing.
X_dev, X_test, y_dev, y_test = train_test_split(
    X_all,
    y,
    test_size=0.15,
    random_state=RANDOM_STATE,
)

# Remaining data: 60% fit, 15% validation, 10% calibration
# of the original dataset.
X_fit, X_temp, y_fit, y_temp = train_test_split(
    X_dev,
    y_dev,
    test_size=25 / 85,
    random_state=RANDOM_STATE,
)

X_val, X_cal, y_val, y_cal = train_test_split(
    X_temp,
    y_temp,
    test_size=0.40,
    random_state=RANDOM_STATE,
)

print("\nData split:")
print(f"Fit:         {len(X_fit):,} rows")
print(f"Validation:  {len(X_val):,} rows")
print(f"Calibration: {len(X_cal):,} rows")
print(f"Test:        {len(X_test):,} rows")

def make_preprocessor():
    """Impute missing values and standardize features."""

    return Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

# ============================================================
# CRISP-DM 4: MODELING
# ============================================================

print("\n" + "=" * 70)
print("CRISP-DM 4: MODELING")
print("=" * 70)
```

```
METHODS = [
    "SelectKBest",
    "RFE",
    "Sequential",
    "Lasso",
    "Permutation",
]

def fit_feature_selector(method, k, X_train, y_train):
    """
    Fit feature selection using training data only.

    Returns:
        List of selected original/engineered feature names.
    """

    n_features = X_train.shape[1]
    k = min(k, n_features)

    preprocessor = make_preprocessor()
    X_scaled = preprocessor.fit_transform(X_train)
    y_array = np.asarray(y_train)

    # ------------------------------------------------------------
    # Method 1: SelectKBest
    # ------------------------------------------------------------
    if method == "SelectKBest":

        selector = SelectKBest(
            score_func=f_regression,
            k=k,
        )

        selector.fit(X_scaled, y_array)
        mask = selector.get_support()

    # ------------------------------------------------------------
    # Method 2: Recursive Feature Elimination
    # ------------------------------------------------------------
    elif method == "RFE":

        selector = RFE(
            estimator=LinearRegression(),
            n_features_to_select=k,
            step=1,
        )

        selector.fit(X_scaled, y_array)
        mask = selector.get_support()

    # ------------------------------------------------------------
    # Method 3: Sequential Feature Selection
    # Forward selection with a training-only subsample.
    # ------------------------------------------------------------
    elif method == "Sequential":

        rng = np.random.RandomState(RANDOM_STATE)
        n_sample = min(len(X_scaled), 3000)

        idx = rng.choice(
            len(X_scaled),
            size=n_sample,
```

```
            replace=False,
        )

        selector = SequentialFeatureSelector(
            estimator=LinearRegression(),
            n_features_to_select=k,
            direction="forward",
            scoring="neg_root_mean_squared_error",
            cv=2,
            n_jobs=-1,
        )

        selector.fit(X_scaled[idx], y_array[idx])
        mask = selector.get_support()

    # ------------------------------------------------------------
    # Method 4: Lasso
    # Rank coefficients by absolute magnitude and select top k.
    # ------------------------------------------------------------
    elif method == "Lasso":

        selector = LassoCV(
            cv=3,
            random_state=RANDOM_STATE,
            max_iter=5000,
            n_jobs=-1,
        )

        selector.fit(X_scaled, y_array)

        importance = np.abs(selector.coef_)
        indices = np.argsort(importance)[::-1][:k]

        mask = np.zeros(n_features, dtype=bool)
        mask[indices] = True

    # ------------------------------------------------------------
    # Method 5: Permutation Importance
    # ------------------------------------------------------------
    elif method == "Permutation":

        baseline = LinearRegression()
        baseline.fit(X_scaled, y_array)

        rng = np.random.RandomState(RANDOM_STATE)
        n_sample = min(len(X_scaled), 10000)

        idx = rng.choice(
            len(X_scaled),
            size=n_sample,
            replace=False,
        )

        perm = permutation_importance(
            baseline,
            X_scaled[idx],
            y_array[idx],
            scoring="neg_root_mean_squared_error",
            n_repeats=3,
            random_state=RANDOM_STATE,
            n_jobs=-1,
        )
```

```
        indices = np.argsort(
            perm.importances_mean
        )[::-1][:k]

        mask = np.zeros(n_features, dtype=bool)
        mask[indices] = True

    else:
        raise ValueError(f"Unknown method: {method}")

    selected = X_train.columns[mask].tolist()

    if len(selected) != k:
        raise RuntimeError(
            f"{method} selected {len(selected)} features, "
            f"but {k} were requested."
        )

    return selected

# ------------------------------------------------------------
# Run all 20 feature-selection experiments
# ------------------------------------------------------------

results = []
selected_features = {}
fitted_models = {}

for method in METHODS:
    for k in FEATURE_COUNTS:

        print(f"\nTraining {method}, k={k}")
        start_time = time.time()

        features = fit_feature_selector(
            method,
            k,
            X_fit,
            y_fit,
        )

        model = Pipeline([
            ("preprocess", make_preprocessor()),
            ("regressor", LinearRegression()),
        ])

        model.fit(
            X_fit[features],
            y_fit,
        )

        val_pred = model.predict(
            X_val[features]
        )

        val_rmse = np.sqrt(
            mean_squared_error(y_val, val_pred)
        )

        val_mae = mean_absolute_error(
            y_val, val_pred
```

```
        )

        val_r2 = r2_score(
            y_val, val_pred
        )

        key = (method, k)

        selected_features[key] = features
        fitted_models[key] = model

        results.append({
            "method": method,
            "requested_k": k,
            "n_features": len(features),
            "validation_RMSE": val_rmse,
            "validation_MAE": val_mae,
            "validation_R2": val_r2,
            "runtime_seconds": time.time() - start_time,
        })

results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    "validation_RMSE",
    ascending=True,
).reset_index(drop=True)

results_df.to_csv(
    OUTPUT_DIR / "feature_selection_comparison.csv",
    index=False,
)

print("\nFeature selection results:")
print(results_df.to_string(index=False))

# Plot comparison of all feature-selection methods.
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.lineplot(
    data=results_df,
    x="n_features",
    y="validation_RMSE",
    hue="method",
    marker="o",
    ax=axes[0],
)

axes[0].set_title("Feature Selection vs Validation RMSE")
axes[0].set_xlabel("Number of Features")
axes[0].set_ylabel("Validation RMSE")

sns.lineplot(
    data=results_df,
    x="n_features",
    y="validation_R2",
    hue="method",
    marker="o",
    ax=axes[1],
)

axes[1].set_title("Feature Selection vs Validation R2")
```

```
axes[1].set_xlabel("Number of Features")
axes[1].set_ylabel("Validation R2")

plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "feature_selection_comparison.png",
    dpi=160,
    bbox_inches="tight",
)
plt.close()

# ============================================================
# CRISP-DM 5: MODEL EVALUATION
# ============================================================

print("\n" + "=" * 70)
print("CRISP-DM 5: MODEL EVALUATION")
print("=" * 70)

# Select the best configuration using validation RMSE only.
best_row = results_df.iloc[0]

best_method = best_row["method"]
best_k = int(best_row["requested_k"])
best_key = (best_method, best_k)

best_features = selected_features[best_key]
best_model = fitted_models[best_key]

print("\nBest configuration:")
print("Method:", best_method)
print("Feature count:", best_k)
print("Features:", best_features)

# ------------------------------------------------------------
# Final test-set evaluation
# ------------------------------------------------------------

test_pred = best_model.predict(
    X_test[best_features]
)

test_rmse = np.sqrt(
    mean_squared_error(y_test, test_pred)
)

test_mae = mean_absolute_error(
    y_test, test_pred
)

test_r2 = r2_score(
    y_test, test_pred
)

print("\nIndependent test metrics:")
print(f"RMSE: {test_rmse:.6f}")
print(f"MAE : {test_mae:.6f}")
print(f"R2  : {test_r2:.6f}")

test_metrics = pd.DataFrame([{
    "best_method": best_method,
    "n_features": best_k,
```

```
    "RMSE": test_rmse,
    "MAE": test_mae,
    "R2": test_r2,
}])

test_metrics.to_csv(
    OUTPUT_DIR / "test_metrics.csv",
    index=False,
)

# ------------------------------------------------------------
# 95% split conformal prediction intervals
# Calibration data is not used to fit the model.
# ------------------------------------------------------------

cal_pred = best_model.predict(
    X_cal[best_features]
)

cal_residuals = np.abs(
    np.asarray(y_cal) - cal_pred
)

n_cal = len(cal_residuals)

rank = int(
    np.ceil((n_cal + 1) * (1 - ALPHA))
)

if rank > n_cal:
    q_hat = np.inf
else:
    q_hat = np.sort(cal_residuals)[rank - 1]

lower = test_pred - q_hat
upper = test_pred + q_hat

test_array = np.asarray(y_test)

coverage = np.mean(
    (test_array >= lower) &
    (test_array <= upper)
)

mean_width = np.mean(upper - lower)

print("\n95% Prediction Interval:")
print(f"Conformal quantile: {q_hat:.6f}")
print(f"Observed coverage:  {coverage:.4%}")
print(f"Mean interval width: {mean_width:.6f}")

interval_metrics = pd.DataFrame([{
    "nominal_coverage": 1 - ALPHA,
    "observed_test_coverage": coverage,
    "mean_interval_width": mean_width,
    "conformal_quantile": q_hat,
}])

interval_metrics.to_csv(
    OUTPUT_DIR / "prediction_interval_metrics.csv",
    index=False,
)
```

```
# ------------------------------------------------------------
# Plot 1: Actual vs predicted with prediction interval
# ------------------------------------------------------------

plot_df = pd.DataFrame({
    "actual": test_array,
    "predicted": test_pred,
    "lower": lower,
    "upper": upper,
})

plot_df = plot_df.sample(
    n=min(300, len(plot_df)),
    random_state=RANDOM_STATE,
)

plot_df = plot_df.sort_values(
    "predicted"
).reset_index(drop=True)

x = np.arange(len(plot_df))

fig, ax = plt.subplots(figsize=(14, 6))

ax.fill_between(
    x,
    plot_df["lower"].to_numpy(),
    plot_df["upper"].to_numpy(),
    alpha=0.25,
    label="95% Prediction Interval",
)

ax.plot(
    x,
    plot_df["actual"].to_numpy(),
    ".",
    markersize=4,
    label="Actual",
)

ax.plot(
    x,
    plot_df["predicted"].to_numpy(),
    "-",
    linewidth=1.5,
    label="Predicted",
)

ax.set_title(
    f"Actual vs Predicted with 95% Prediction Interval\n"
    f"{best_method}, k={best_k}"
)

ax.set_xlabel("Test samples sorted by predicted value")
ax.set_ylabel("FloodProbability")
ax.legend()

plt.tight_layout()
plt.savefig(
    OUTPUT_DIR / "prediction_interval.png",
    dpi=160,
```

```
    bbox_inches="tight",
)
plt.close()

# ------------------------------------------------------------
# Plot 2: Parity plot and residual plot
# ------------------------------------------------------------

residuals = test_array - test_pred

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(
    test_array,
    test_pred,
    alpha=0.2,
    s=8,
)

vmin = min(
    float(np.min(test_array)),
    float(np.min(test_pred)),
)

vmax = max(
    float(np.max(test_array)),
    float(np.max(test_pred)),
)

axes[0].plot(
    [vmin, vmax],
    [vmin, vmax],
    "k--",
    label="Perfect Prediction",
)

axes[0].set_title(
    f"Actual vs Predicted (R2={test_r2:.4f})"
)

axes[0].set_xlabel("Actual FloodProbability")
axes[0].set_ylabel("Predicted FloodProbability")
axes[0].legend()

axes[1].scatter(
    test_pred,
    residuals,
    alpha=0.2,
    s=8,
)

axes[1].axhline(
    0,
    linestyle="--",
    color="black",
)

axes[1].set_title("Residual Analysis")
axes[1].set_xlabel("Predicted FloodProbability")
axes[1].set_ylabel("Residual")

plt.tight_layout()
```

```
plt.savefig(
    OUTPUT_DIR / "regression_diagnostics.png",
    dpi=160,
    bbox_inches="tight",
)
plt.close()

# ============================================================
# CRISP-DM 6: DEPLOYMENT
# ============================================================

print("\n" + "=" * 70)
print("CRISP-DM 6: DEPLOYMENT")
print("=" * 70)

# Save the selected features.
with open(
    OUTPUT_DIR / "best_features.json",
    "w",
    encoding="utf-8",
) as file:
    json.dump(
        {
            "method": best_method,
            "n_features": best_k,
            "features": best_features,
        },
        file,
        ensure_ascii=False,
        indent=2,
    )

# Save the model evaluated on the held-out test set.
joblib.dump(
    best_model,
    OUTPUT_DIR / "best_model.joblib",
)

# Refit the selected configuration on all labeled training data
# for the final Kaggle submission.
X_submission = add_interactions(
    train_df[base_features].copy()
)

X_kaggle_test = add_interactions(
    test_df[base_features].copy()
)

submission_model = Pipeline([
    ("preprocess", make_preprocessor()),
    ("regressor", LinearRegression()),
])

submission_model.fit(
    X_submission[best_features],
    train_df[TARGET],
)

kaggle_pred = submission_model.predict(
    X_kaggle_test[best_features]
)
```

```
if SUBMISSION_PATH.exists():

    submission = pd.read_csv(SUBMISSION_PATH)

    if len(submission) != len(test_df):
        raise ValueError(
            "sample_submission.csv row count does not match test.csv."
        )

    submission[TARGET] = kaggle_pred

else:

    submission = pd.DataFrame({
        TARGET: kaggle_pred,
    })

    if ID_COL is not None and ID_COL in test_df.columns:
        submission.insert(
            0,
            ID_COL,
            test_df[ID_COL].to_numpy(),
        )

submission.to_csv(
    OUTPUT_DIR / "submission.csv",
    index=False,
)

joblib.dump(
    submission_model,
    OUTPUT_DIR / "submission_model.joblib",
)

# Save experiment configuration.
config = {
    "target": TARGET,
    "candidate_feature_count": int(X_all.shape[1]),
    "feature_counts": FEATURE_COUNTS,
    "methods": METHODS,
    "best_method": best_method,
    "best_k": best_k,
    "random_state": RANDOM_STATE,
    "prediction_interval_alpha": ALPHA,
}

with open(
    OUTPUT_DIR / "experiment_config.json",
    "w",
    encoding="utf-8",
) as file:
    json.dump(
        config,
        file,
        ensure_ascii=False,
        indent=2,
    )

print("\nAll tasks completed.")
print("Output directory:", OUTPUT_DIR.resolve())
print("\nGenerated files:")
```

```
for path in sorted(OUTPUT_DIR.iterdir()):
    print(" -", path.name)

print("\nFinal Kaggle submission preview:")
print(submission.head())

print("\nFinal test metrics:")
print(test_metrics.to_string(index=False))

print("\nPrediction interval metrics:")
print(interval_metrics.to_string(index=False))
```

執行完成後的輸出

`feature_selection_comparison.csv`

20 組方法與特徵數量的驗證集 RMSE、MAE、R^2 及執行時間。

`prediction_interval.png`

實際值、預測值與 95% 預測區間。

`regression_diagnostics.png`

預測值對實際值的比較，以及殘差分析。

`test_metrics.csv`、**`prediction_interval_metrics.csv`**

獨立測試集評估與預測區間覆蓋率。

`best_model.joblib`、**`submission.csv`**

保留測試集評估模型，並產生供 Kaggle 提交的預測檔案。

執行時的三個注意事項

1. **特徵數量：** 程式預期原始資料有 20 個數值特徵，再加入 10 個交互作用特徵，合計 30 個。如果欄位名稱不一致，請依照你的 CSV 調整 `INTERACTIONS`。
2. **計算時間：** `Sequential Feature Selection` 最耗時，因為它需要反覆比較候選特徵。若執行太慢，可以降低程式中的 `n_sample`。
3. **預測區間：** 95% 是理論上的邊際覆蓋目標，實際測試集覆蓋率可能高於或低於 95%；請以程式輸出的 `observed_test_coverage` 和 `mean_interval_width` 報告實際結果。

另外，這份程式以驗證集 RMSE 選擇最佳組合，並以測試集 R^2、RMSE、MAE 做最終評估；由於 Kaggle 官方主要看 R^2，如果你希望完全依照官方指標選模型，可將 `results_df` 的排序改為 `validation_R2` 由大到小。